The Geometry of Proof: Linear and Non-Linear Representations in a Formal Language Model

Nathan Hoehndorf
Colorado School of Mines

May 2026

Abstract

Does a language model trained on formal mathematical proofs merely learn to predict the next token, or does it develop an internal "map" of logical properties? We investigate the Linear Representation Hypothesis (LRH) within the context of ReProver, a retrieval-augmented encoder-decoder model trained on Lean 4. By probing the residual stream of the ReProver encoder across all 12 layers, we demonstrate that abstract logical features—specifically the presence of inductive structures and the distinction between equality versus inequality—emerge as linearly separable directions in activation space with substantial improvements over majority-class baselines. The `equality_vs_inequality` feature achieves 88% accuracy at Layer 7, a +29.4 percentage point lift over baseline, while `inductive_structure` peaks near 84% at Layer 6. Probing for the raw count of variables (`variable_count`) via regression, however, yields performance that closely tracks a character-count heuristic baseline, suggesting that quantitative properties may be stored in non-linear or distributed representations. PCA of the residual stream reveals a "compression-expansion-convergence" cycle across layers: tight syntactic clusters in early layers, expanded and more geometrically expressive structure in the middle layers (6–8), and a compressed "comet-like" configuration at the final layer. These results support the view that the middle-to-late layers of a formal-language model can synthesize some structural properties of logic into lower-dimensional space before converging on tactic predictions.

1 Introduction

The Linear Representation Hypothesis (LRH) proposes that large language models (LLMs) represent high-level semantic concepts as directions in a high-dimensional vector space [Park et al., 2023; Elhage et al., 2021]. Evidence for this view has accumulated across a variety of domains: sentiment polarity, grammatical number, spatial relationships, and factual associations have all been shown to correspond to approximately linear directions in transformer activation spaces [Park et al., 2023; Zou et al., 2023]. If this hypothesis generalizes to formal logic, it carries significant implications both for mechanistic interpretability and for the practical goal of building more reliable automated theorem provers.

Formal mathematical systems offer a uniquely clean laboratory for testing the LRH. Unlike natural language, where concepts are inherently graded and ambiguous, Lean 4's type theory gives every tactic state a precise syntactic and semantic structure. The presence of a universal quantifier, the sign of a relation, the use of an inductive type—these are not fuzzy concepts but exact, verifiable properties of the proof state. If a model trained on Lean 4 proofs encodes these properties linearly, we would expect a simple linear classifier trained on intermediate activations to surpass majority-class baselines by a meaningful margin.

We investigate this question using ReProver [Yang et al., 2023], a retrieval-augmented model built on ByT5 [Xue et al., 2022] and fine-tuned on the Lean 4 mathlib4 library. ReProver's encoder processes the current proof state—a Lean 4 tactic state string—and produces a sequence of contextual embeddings used for premise retrieval and tactic generation. We extract activations from each of the encoder's 12 layers and train linear probes to predict three binary logical features (`quantifier_presence`, `equality_vs_inequality`, `inductive_structure`) and one regression target (`variable_count`). Our central findings are:

1. **Binary logical categories are linearly separable in middle layers.** `equality_vs_inequality` and `inductive_structure` peak in the Layer 6–8 range, with +28–30 percentage point improvements over majority-class baselines.
2. **Quantitative counting is not well-represented linearly.** The `variable_count` probe only modestly outperforms a simple character-count baseline, suggesting limits of the LRH for ordinal/cardinal properties.
3. **PCA reveals a compression-expansion-convergence geometry** that tracks the layer-by-layer development of logical structure.

The remainder of the paper is organized as follows. Section 2 reviews related work on probing transformers and on neural theorem proving. Section 3 describes our methodology in detail. Section 4 presents experimental results, and Section 5 discusses their interpretation. Section 6 concludes with directions for future work.

2 Related Work

2.1 The Linear Representation Hypothesis

Elhage et al. [2021] proposed a mathematical framework for analyzing transformer circuits, laying a foundation for understanding how information is stored and transformed across layers. Their work established that residual stream activations can be decomposed into interpretable components. Subsequent empirical work has found that individual neurons and directions in transformer activation spaces correspond to identifiable semantic properties [Geva et al., 2021; Meng et al., 2022].

The LRH received more explicit formulation in work on "representation engineering" [Zou et al., 2023] and the concept of linear representations [Park et al., 2023], which showed that a range of high-level semantic concepts—including sentiment, truthfulness, and emotional valence—have approximately linear representations in LLMs. The probing methodology we adopt follows the tradition of Alain and Bengio [2017], who trained linear classifiers on intermediate layer representations to measure the "linear separability" of a concept at each layer depth.

2.2 Mechanistic Interpretability in Formal Domains

Most probing work has focused on natural language. Relatively little work has applied these techniques to models trained on formal languages. An exception is work on code language models, where researchers have found that properties like variable scope, data type, and syntactic role can be recovered from intermediate activations [Karmakar et al., 2021].

Formal mathematical systems provide advantages over natural language and code as a testing ground: the ground truth labels are exact, unambiguous, and can be extracted automatically by parsing the Lean 4 syntax tree.

2.3 Lean 4 and Formal Proof States

Lean 4 is an interactive theorem prover and functional programming language grounded in dependent type theory. In Lean 4, a mathematical proof is literally a program: the type system enforces logical validity, so a well-typed term is a correct proof by construction. The central object of interest for our experiments is the tactic state, which is what Lean displays to the user at any intermediate step of a proof. A tactic state has the form:

```
h : P
h : Q
 R
```

where the lines above the turnstile () are hypotheses — things the prover currently knows — and the line below is the goal — what remains to be proved. The user applies tactics, which are commands (e.g., induction, simp, rw, exact) that transform the tactic state step by step until the goal is discharged. This makes the tactic state a self-contained, syntactically precise snapshot of the proof's logical status at a given moment. The features we probe — presence of quantifiers, the relational sign of the goal, the involvement of inductive types — are all directly readable from the tactic state string's syntax tree and carry unambiguous semantic content. This is in sharp contrast to natural language probing targets like sentiment or factual associations, which are inherently graded. mathlib4 is the community-maintained Lean 4 library containing thousands of formalized theorems spanning algebra, topology, number theory, and analysis; it serves as our data source.

2.4 Neural Theorem Proving

Neural theorem proving has advanced substantially with models like GPT-f [Polu & Han, 2020], Hypertree Proof Search [Lample et al., 2022], and ReProver [Yang et al., 2023]. ReProver specifically introduced a retrieval-augmented generation architecture that uses a dense retrieval encoder to identify relevant lemmas from mathlib4, feeding them as context to the tactic generator. The encoder is built on ByT5 [Xue et al., 2022], a byte-level T5 variant, enabling the model to operate directly on the raw character sequence of a Lean 4 tactic state without a tokenizer—a design choice particularly suitable for formal syntax where character-level structure carries meaning.

2.5 Probing Methodology

Our probing approach follows established practices in the NLP literature [Tenney et al., 2019; Hewitt & Manning, 2019]. A "probe" is a simple classifier (typically linear) trained on frozen representations from a specific layer. High probe accuracy is taken as evidence that the corresponding concept is linearly encoded at that layer, while performance tracking the layer depth reveals how the concept emerges and dissolves across the forward pass. We use binary cross-entropy probes for categorical features and mean-squared-error regression probes for continuous targets, following conventions in the mechanistic interpretability literature.

3 Methodology

3.1 Model and Architecture

We use the ReProver encoder, based on the ByT5 architecture [Xue et al., 2022]. ByT5 processes raw UTF-8 bytes rather than subword tokens, making it naturally suited to the formal syntax

of Lean 4, where individual characters and punctuation carry semantic significance. The encoder consists of 12 transformer layers. We probe the residual stream output (`hook_resid_post`) at each layer, yielding 12 activation tensors per input sample.

3.2 Dataset

Our dataset consists of Lean 4 tactic states extracted from proofs in the mathlib4 library. The models were run on 2500 sample Lean states, extracted from the lean-community Github page. We applied an 80/20 train-test split for all probing experiments.

3.3 Feature Engineering

From each tactic state string, we extract the following features:

Binary features (classification targets):

- `quantifier_presence`: Whether the tactic state contains a universal ($\forall$) or existential ($\exists$) quantifier.
- `equality_vs_inequality`: Whether the primary relation in the goal is an equality (`=`) as opposed to an inequality ($\leq$, `<`, $\geq$, `>`).
- `inductive_structure`: Whether the proof state involves an inductive type or inductive hypothesis (e.g., presence of `ih`, `Nat.rec`, or similar constructs).

Regression target:

- `variable_count`: The number of distinct free variables appearing in the tactic state, extracted by a Python script that ran through the data, counted the number of free variables, and then labeled the samples accordingly.

Majority-class baselines for the binary features are:

`quantifier_presence`	61.46%,
`equality_vs_inequality`	58.56%,
`inductive_structure`	53.97%.

3.4 Activation Extraction

For each input tactic state, we pass the string through the ReProver encoder and extract the residual stream at the final token position of the input at each of the 12 layers. Following established heuristics in mechanistic interpretability, we use the final token position as a natural aggregation point: in transformer sequence models, the final position accumulates contextual information from all preceding tokens through the residual stream and attention mechanism, making it a natural bottleneck for downstream feature readout [Elhage et al., 2021]. During extraction, activations were computed in evaluation mode with gradient tracking disabled and cached layer-by-layer to avoid redundant forward passes. Inputs were processed in mini-batches sized to fit GPU memory constraints, and the resulting residual stream tensors were serialized to disk for downstream probing experiments.

3.5 Probe Architecture and Training

For binary features, we train a `torch.nn.Linear` layer (input dim = 512, output dim = 1) with `BCEWithLogitsLoss` using the Adam optimizer (lr = 0.001) for 101 epochs. For the regression target (`variable_count`), we train a linear layer with `MSELoss`. All probes are trained on the frozen activations from a given layer; the encoder weights are not updated.

To normalize the regression error, we report the MSE after normalized via subtracting mean and dividing by standard deviation. The character-count baseline for regression is computed by training a linear regression from the raw character length of the tactic state string to `variable_count`, providing a lower bound on what a semantics-free model could achieve.

3.6 Control Tasks

To validate that probe accuracy reflects semantic content rather than training set memorization or class imbalance, we implement:

- **Random label controls**: We shuffle the labels to see if the model performs randomly, and they all achieve a 0.50.
- **Character-count baseline**: For `variable_count`, a regression from string length serves as a syntax-only baseline.

3.7 Geometric Analysis

We complement the probing results with principal component analysis (PCA) of the residual stream activations at each layer. For each layer, we project the 500 test-set activations onto the top 3 principal components and visualize the resulting point cloud from three viewing angles. This analysis reveals how the geometry of the representation space evolves across the network.

The codebase for this project is available at https://github.com/nathanhoehndorf/lean-interp.

4 Results

4.1 Binary Feature Probing

Table 1: Peak probe accuracy by feature and layer, with majority-class baselines.

Feature	Baseline	Peak Accuracy	Peak Layer	Lift
`equality_vs_inequality`	58.56%	88.0%	Layer 7	+29.4pp
`quantifier_presence`	61.46%	80.0%	Layer 7	+18.5pp
`inductive_structure`	53.97%	84.0%	Layer 6	+30.0pp

All three binary features demonstrate substantial improvement over their majority-class baselines, confirming that the ReProver encoder linearly encodes syntactic-semantic distinctions in formal Lean 4 proof states.

The clearest signal comes from `equality_vs_inequality`, which rises from 75% at Layer 0 to a peak of 88% at Layer 7 before stabilizing around 84% through the final layer. This suggests that the distinction between relational sign is partially recoverable from the embeddings but sharpens substantially through the middle layers. `inductive_structure` shows the most dramatic early-layer suppression—starting at only 63% (barely above baseline) at Layer 0 and Layer 1, then

rising sharply to 84% at Layer 6. This is consistent with the view that inductive structure is a higher-order logical property requiring several layers of processing to emerge as a linearly readable feature. `quantifier_presence` is more uniformly encoded, staying in the 71–80% range throughout the network, reaching its peak also at Layer 7.

The binary accuracy heatmap (Figure 1) shows the full layer-by-layer profile. The Layer 6–8 region consistently achieves the highest accuracy across all three features, suggesting this is the "semantic synthesis" zone of the network.

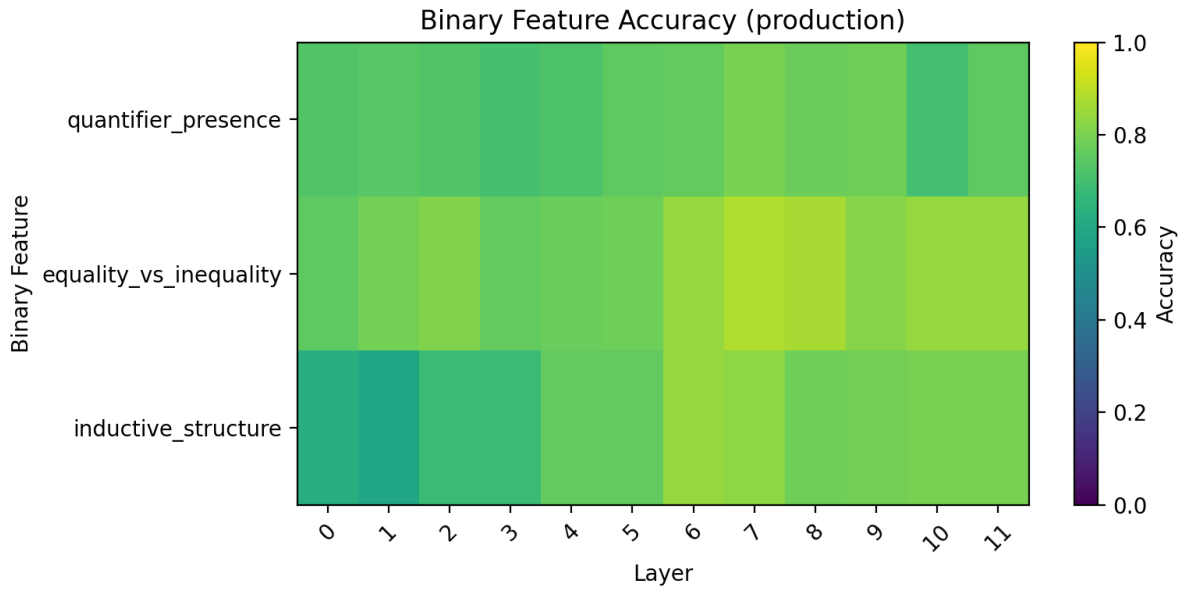

Figure 1: Layer-wise probe accuracy for binary logical features. Probe performance consistently peaks in Layers 6–8, suggesting that the middle layers of the encoder are responsible for synthesizing abstract logical structure.

4.2 Regression Probe: Variable Count

For the regression task, we report normalized MSE (lower is better). The `variable_count` probe achieves its minimum MSE of approximately 0.277 at Layer 4, compared to the character-count baseline which reaches its own minimum of around 0.198 at Layer 1.

Crucially, the two curves track each other closely throughout the network and are not consistently separated by a large margin.

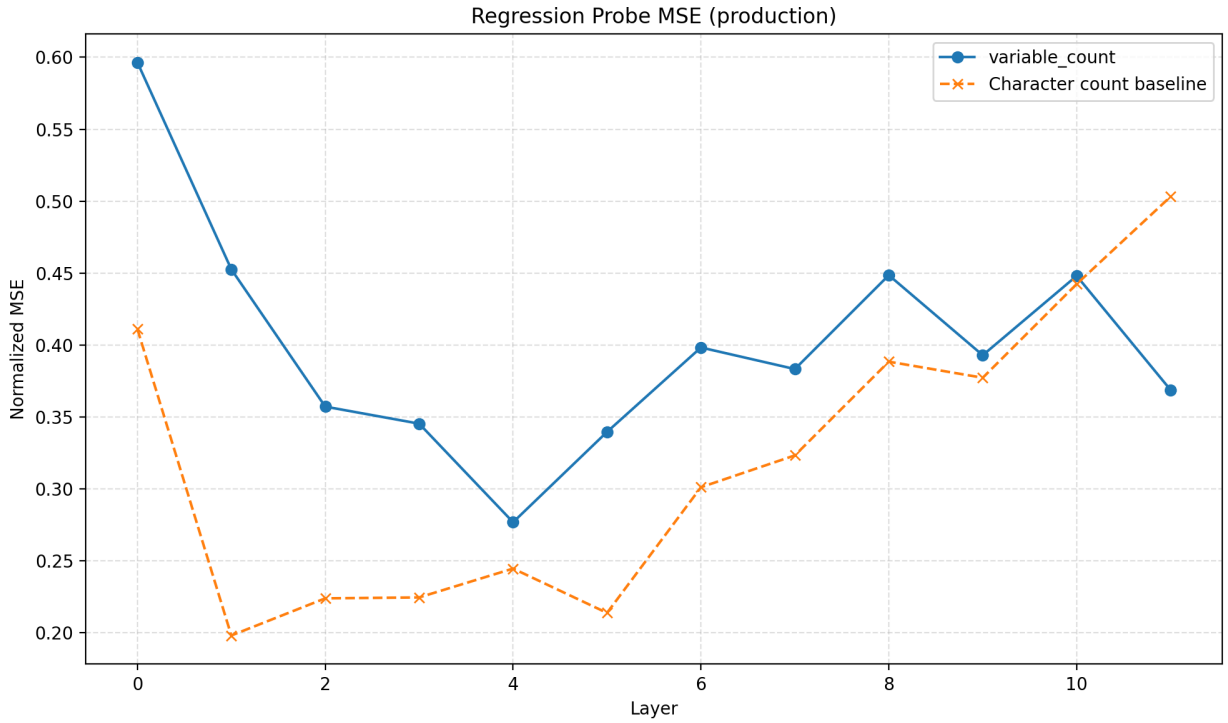

Figure 2: Normalized MSE of the variable-count regression probe across layers compared to a character-count baseline. The close tracking between the two curves suggests that the probe primarily exploits length-correlated information rather than a semantically meaningful counting representation.

In the later layers (8–11), the probe and baseline curves nearly converge. This indicates that the probe is not learning to count variables in any semantically rich sense—rather, it is leveraging the same length-correlated signal that the character-count baseline uses. The model does not appear to linearly represent exact variable count as a scalar.

This stands in contrast to the binary probing results and represents a potential limitation of the LRH in this domain: categorical logical properties are linearly encoded; ordinal/cardinal ones may not be.

4.3 Feature Independence

The correlation matrix over the four features (Figure 3) shows near-zero inter-feature correlations: `equality_vs_inequality` and `quantifier_presence` have $r = -0.06$, `inductive_structure` and `equality_vs_inequality` have $r = 0.07$, and `variable_count` correlates weakly with `quantifier_presence` at $r = 0.19$. This validates that the probes are recovering independent directions in activation space and that the accuracy results are not artifacts of features being collinear.

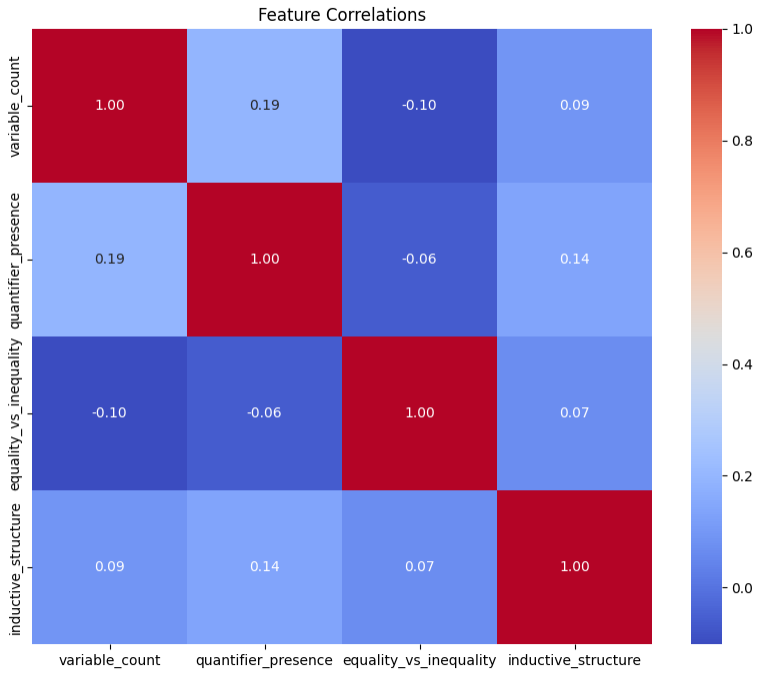

Figure 3: Correlation matrix between extracted logical features. Most feature pairs exhibit near-zero correlation, suggesting that probe performance is not driven by feature collinearity.

4.4 Geometric Evolution of Activation Space (PCA)

The PCA plots (Figures 4–7) reveal a rich geometric evolution across layers.

Early layers (0–2): The point cloud shows several discrete, tight clusters. This structure likely reflects syntactic surface patterns in the Lean 4 byte sequence—e.g., different proof state templates cluster based on their character-level similarity. The clusters are well-separated in PC1–PC3 space but interleaved between train (blue) and test (orange) points, suggesting the model has not yet built semantically meaningful geometry.

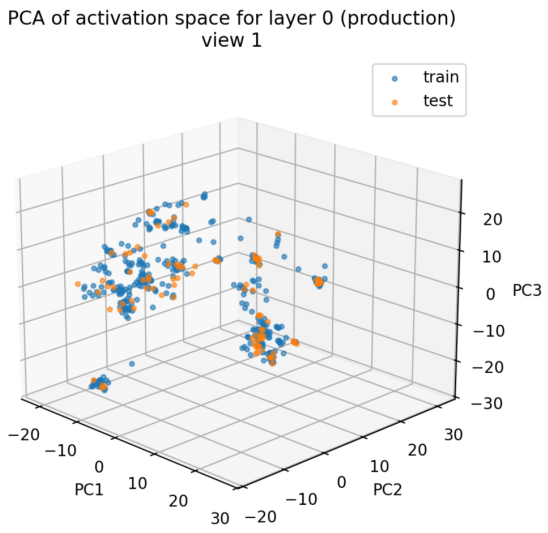

Figure 4: Layer 0 3-D Principal Component Analysis.

Middle layers (3–6): The tight clusters gradually dissolve, and the activation cloud expands. By Layer 5, the data occupies a broader, more diffuse region of PC space. This "expansion" phase is characteristic of the middle layers building more abstract, distributed representations. The range of the principal components increases, indicating that the model is writing richer information into the residual stream.

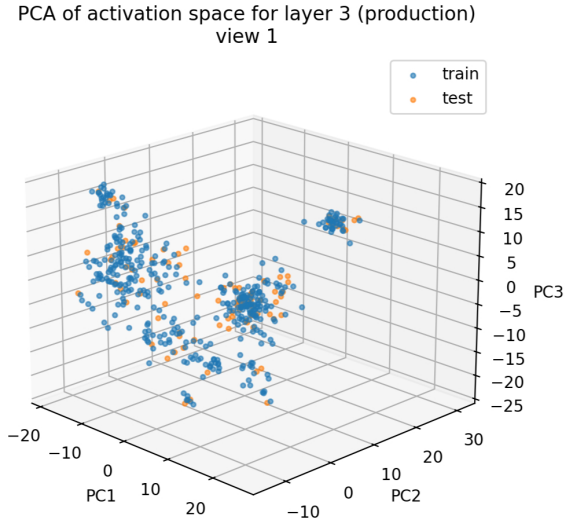

Figure 5: Layer 3 3-D Principal Component Analysis.

Peak semantic layers (6–8): At Layers 7 and 8, the distribution reaches its maximum spread and most structurally complex form. This corresponds precisely to the layers where binary probe accuracy is highest, supporting the interpretation that the model is actively encoding abstract logical features in these layers.

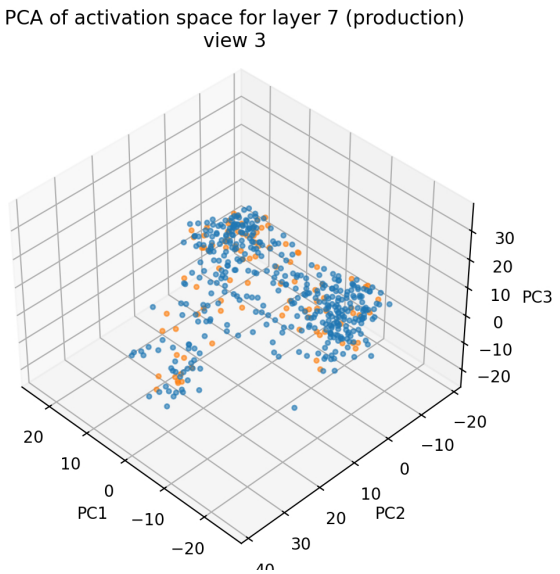

Figure 6: Layer 7 3-D Principal Component Analysis.

Late layers (9–11): The cloud begins to converge and compress. By Layer 11, the activations collapse into a dense, elongated "comet-like" structure with a compact body and a trailing tail of outliers. This convergent geometry is consistent with the encoder preparing a compact, task-specific representation to be passed to the decoder for tactic generation, which is hypothesized due to how intuitive the mechanics would be.

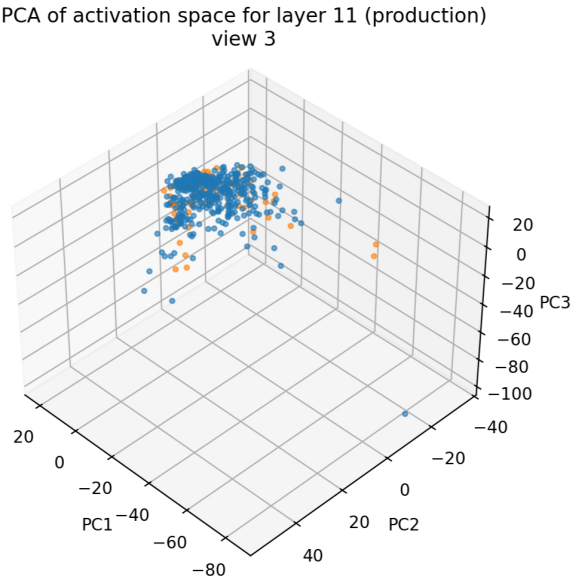

Figure 7: Layer 11 3-D Principal Component Analysis.

5 Discussion

5.1 The "Layer 8 Spike" and Semantic Synthesis

The convergence of peak probe accuracy at Layers 6–8 for all three binary features—independent of each other and robust across widely different baselines—is the central finding of this work. It is consistent with the view that the ReProver encoder could be transitioning from syntactic to semantic processing in its middle layers, and that this transition produces a representation space where abstract logical properties become linearly readable. This mirrors findings in natural language models, where middle layers tend to encode syntactic and semantic structure while early and late layers are more strongly tied to surface form and output logits, respectively [Tenney et al., 2019].

5.2 Limits of the Linear Representation Hypothesis

The failure of the `variable_count` probe to significantly outperform the character-count baseline is theoretically interesting. Variable counting is an ordinal/cardinal property, not a categorical one. It is possible that the model encodes variable count through a population code—a combination of many dimensions—rather than a single linear direction. Alternatively, the model may not represent exact variable count at all, instead relying on approximate length-correlated signals when needed. Future work employing non-linear probes (e.g., MLP probes) or sparse representation analyses could distinguish these hypotheses.

5.3 Caveats and Limitations

Several caveats deserve mention. First, probing accuracy is a necessary but not sufficient condition for the LRH: a linear probe can succeed even if the concept is encoded in a distributed, entangled fashion that is not causally relevant to the model's behavior. Second, we probe only the final token position; it is possible that other positions carry complementary information about these features.

5.4 What Went Wrong, What Went Right

The clearest success is the binary probing pipeline: the layer-by-layer accuracy profiles are clean, interpretable, and consistent with theoretical predictions. The PCA analysis corroborates the probing results with qualitative geometric evidence.

The regression probe is less conclusive. In retrospect, normalizing MSE by the variance of the target (rather than using raw MSE) would have made the baseline comparison cleaner. Additionally, more careful feature engineering for `variable_count`—for instance, binning it into ordinal brackets—might have converted this into a classification problem where linear probes are more appropriate.

6 Conclusion and Future Work

We have demonstrated that the ReProver encoder, a ByT5-based model trained on Lean 4 proofs from mathlib4, develops linearly separable internal representations of abstract logical properties including equality/inequality distinction and inductive structure. These features peak in linear readability at Layers 6–8 of the 12-layer encoder, consistent with the view that middle layers perform semantic synthesis. In contrast, the exact count of variables is not well-captured by a linear probe, suggesting limits of the LRH for quantitative properties.

Several directions follow naturally from this work:

Activation steering. If the `equality_vs_inequality` direction can be reliably identified and isolated, one could perform activation steering: adding a scaled version of this direction to the residual stream to shift the model's internal representation toward inequality-dominated proof states, then measuring whether tactic generation changes accordingly. This would provide causal evidence that the linear direction is not merely a correlate but an active computational component.

Broader feature vocabulary. Future work could probe for more abstract properties: commutativity, associativity, topological invariance, or the proof-theoretic depth of the current goal. As these concepts are increasingly abstract, we would predict that probes would peak in later layers, providing a test of the "concept abstraction gradient" hypothesis.

Cross-model comparison. Testing whether similar linear structure appears in non-specialized models (e.g., general-purpose code LLMs) versus domain-specific ones (ReProver) would clarify whether this geometry is an artifact of mathlib4 training or a more general property of transformer processing of formal language.

Non-linear probing. To better characterize the `variable_count` failure, future work should employ MLP probes or other non-linear diagnostic tools to determine whether the information is present but non-linearly encoded, versus genuinely absent from the representation.

Bibliography

- Alain, G., & Bengio, Y. (2017). Understanding intermediate layers using linear classifier probes. *ICLR Workshop*.
- Elhage, N., Nanda, N., Olsson, C., Henighan, T., Joseph, N., Mann, B., ... & Olah, C. (2021). A mathematical framework for transformer circuits. *Transformer Circuits Thread*.
- Geva, M., Schuster, R., Berant, J., & Levy, O. (2021). Transformer feed-forward layers are key-value memories. *EMNLP 2021*.
- Hewitt, J., & Manning, C. D. (2019). A structural probe for finding syntax in word representations. *NAACL 2019*.
- Lample, G., Lacroix, T., Lachaux, M. A., Rodriguez, A., Hayat, A., Lavril, T., ... & Martinet, X. (2022). Hypertree proof search for neural theorem proving. *NeurIPS 2022*.
- Meng, K., Bau, D., Andonian, A., & Belinkov, Y. (2022). Locating and editing factual associations in GPT. *NeurIPS 2022*.
- Park, K., Choe, Y. J., & Veitch, V. (2023). The linear representation hypothesis and the geometry of large language models. *arXiv:2311.03658*.
- Polu, S., & Han, J. M. (2020). Generative language modeling for automated theorem proving. *arXiv:2009.03393*.
- Tenney, I., Das, D., & Pavlick, E. (2019). BERT rediscovers the classical NLP pipeline. *ACL 2019*.

- Xue, L., Barua, A., Constant, N., Al-Rfou, R., Narang, S., Kale, M., ... & Raffel, C. (2022). ByT5: Towards a token-free future with pre-trained byte-to-byte models. *TACL*.
- Yang, K., Swope, A. M., Gu, A., Chalamala, R., Song, P., Yu, S., ... & Anandkumar, A. (2023). LeanDojo: Theorem proving with retrieval-augmented language models. *NeurIPS 2023*.
- Zou, A., Phan, L., Chen, S., Campbell, J., Guo, B., Ren, R., ... & Hendrycks, D. (2023). Representation engineering: A top-down approach to AI transparency. *arXiv:2310.01405*.
- Karmakar, A., & Robbes, R. (2021). What do pre-trained code models know about code? *arXiv:2109.01523*.